📄 Advanced RAG System Architecture

---

1. System Overview

This project implements a **production-grade Adaptive Retrieval-Augmented Generation (RAG) system** designed to dynamically select the most efficient response strategy based on query complexity.

The architecture integrates:

- Adaptive RAG (Query Routing)
- Corrective RAG (CRAG – Retrieval Validation)
- Self-RAG (Hallucination and relevance control)
- Hybrid Retrieval (Dense + Sparse + Reranking)
- SQLite-based Caching Layer

The system ensures:

- High accuracy
- Reduced latency
- Grounded and verifiable response

2. End-to-End Workflow

User Query
↓
Cache Check (SQLite)
↓
IF Cache Hit → Return Cached Response → END
↓
IF Cache Miss →
↓
Query Router (A / B / C Classification)
↓
A → Direct LLM Response
B → Retrieval + Validation (CRAG)
C → Multi-hop Agentic Retrieval
↓
Self-RAG Validation Layer
↓
Cache Store (Query + Response + Metadata)
Final Response / Retry / Fail-safe

---

3. Ingestion Pipeline

Tool Used: Docling

Docling is used for ingestion due to its ability to process multiple document formats such as PDF, Word, and HTML, and convert them into structured outputs.

Format Selection: JSON

JSON is selected as the primary format because it preserves:

- Document hierarchy (sections, headings)
- Tables and structured content
- Metadata

This enables better downstream processing, especially for chunking and retrieval.

Duplicate Detection (File-Level)

To ensure data integrity and avoid redundant processing:

- A hash (e.g., SHA-256) is generated for each file
- The hash is compared with previously ingested files
- Duplicate files are skipped

Benefits:

- Avoids re-embedding
- Reduces storage cost
- Improves ingestion efficiency

---

4. Chunking Strategy

Approach: Structure-Aware Parent-Child Chunking

Chunking is performed using the structured JSON output to preserve semantic boundaries such as:

- Headings
- Paragraphs
- Tables

Key Features:

- Maintains contextual integrity
- Prevents semantic fragmentation
- Improves embedding quality

Overlap Strategy:

- Small overlap between chunks
- Ensures context continuity
- Improves retrieval recall

---

5. Vector Storage Layer

Database: Qdrant

Qdrant is used as the vector database for storing embeddings and metadata.

Key Features:

- HNSW indexing for fast similarity search
- Scalable and production-ready
- Supports hybrid search

Stored Data Structure:

```
{
  "id": uuid,
  "text": chunk_text,
  "embedding": vector,
  "metadata": {
    "parent_id": "",
    "parent_title": "",
    "chunk_index": "",
    "document": "",
    "page_number": ""
  }
}
```

---

6. Retrieval System (Hybrid + Reranking)

A multi-stage retrieval pipeline is implemented to improve accuracy.

Step 1: Dense Retrieval

- Uses embeddings stored in Qdrant
- Captures semantic similarity

Step 2: Sparse Retrieval

- BM25-based keyword search
- Captures exact term matching

Step 3: Fusion

- Reciprocal Rank Fusion (RRF)
- Combines dense + sparse results

Step 4: Reranking

- CrossEncoder model (`ms-marco-MiniLM-L-6-v2`)
- Refines top results using deep relevance scoring

---

7. Adaptive RAG (Query Routing Layer)

The system classifies queries into three categories:

Type A – Simple Queries

- Answered directly using LLM
- No retrieval required
- Low latency

Type B – Moderate Queries

- Requires external knowledge
- Uses single-step RAG pipeline

Type C – Complex Queries

- Multi-hop reasoning required
- Uses iterative retrieval and query refinement

Routing Enhancements

1. **Rule-Based Fallback**
    - Simple queries (greetings, general knowledge) routed directly to Type A
2. **LLM-Based Classification**
    - Determines query complexity (A/B/C)
3. **Fail-Safe Mechanism**
    - If classification fails → default to Type B

---

8. Execution Flows(graph.py)

A — Direct Response Flow

```
route_query → simple_generate → END
```

- Fastest path
- No retrieval

B — Corrective RAG (CRAG)

```
retrieve → grade_documents → generate → self_rag_check
```

Key Features:

- Document grading using LLM
- Ensures retrieved documents are relevant

Conditional Logic:

- If documents are relevant → proceed to generation
- If not → trigger web search fallback

Web Search Fallback:

- Query rewriting
- External search
- Response generation using updated context

C — Multi-Hop Agentic Flow

```
generate_subquery → retrieve → grade → generate → self_rag_check
        ↑_____________________________________________|
```

Features:

- Iterative reasoning
- Sub-query generation
- Multiple retrieval cycles

Controls:

- Maximum iteration limit (int - 6)
- Prevents infinite loops

---

9. Self-RAG (Validation Layer)

This layer ensures response quality before final output.

Checks Performed:

1. **Hallucination Detection**
    - Verifies if response is grounded in retrieved data
2. **Relevance Check**
    - Ensures answer is useful and aligned with query

Decision Outcomes:

- "Supported" → Accept response
- "Not Supported" → Retry
- "Not Useful" → Retry

Retry Strategy:

```
IF not useful/supported:
    IF max attempts reached → final_fail
    ELSE IF complexity C → generate_subquery
    ELSE → transform_query
```

Important Behavior:

- The system does NOT return unverified answers
- Retries until:

    - A grounded answer is found OR
    - Maximum attempts are reached

---

10. Caching Layer (SQLite)

A caching layer using SQLite has been implemented to improve performance and reduce cost.

Cache Types:

1. **Query Cache**
    - Stores previously asked queries and responses
    - Avoids redundant LLM calls
2. **Embedding Cache**
    - Stores computed embeddings
    - Prevents recomputation

Benefits:

- Reduced latency
- Lower API cost
- Faster repeated query handling
- Lightweight and easy to maintain

---

11. Model Infrastructure

All LLM operations are handled via:

- LiteLLM (Unified API layer)
- Vertex AI (Model hosting)

Used For:

- Query routing
- Document grading
- Query rewriting
- Response generation

---

12. Fail-Safe Mechanisms

Multiple safeguards are implemented:

1. Rule-based routing fallback
2. LLM fallback (default to Type B)
3. Self-RAG validation
4. Retry loops
5. Final fallback response on failure

---

13. Frontend Architecture (React + Vite + TypeScript)

The frontend is developed using **React with Vite and TypeScript**, providing a fast and interactive interface for interacting with the Adaptive RAG system.

Core Responsibilities

1. File Upload (Ingestion)

- Uploads documents via:

  ```
  POST /ingest
  ```

- Displays upload status and duplicate detection feedback

---

2. Query Submission

- Sends user queries to:

  ```
  POST /query
  ```

- Displays:
  - Generated response
  - Query type (A / B / C)
  - Retrieved context (if applicable)

---

3. Cache Management (SQLite)

- Clear cache:

  ```
  POST /cache/clear
  ```

- Check cache status:

  ```
  GET /cache/status
  ```

- Displays cache hit/miss and stored query information

---

4. Decision Trace Visualization

- Shows internal pipeline steps:
    - Query routing decision
    - Retrieval and grading
    - Self-RAG validation
    - Retry attempts

---

5. Latency Monitoring

- Displays response time for each query
- Helps evaluate system performance and caching efficiency

---

14. Conclusion

This architecture provides a robust, scalable, and intelligent RAG system capable of:

- Dynamically adapting to query complexity
- Ensuring response accuracy and grounding
- Optimizing performance using caching and hybrid retrieval
- Supporting real-world production deployment scenarios